

Stock Forecasting & Portfolio Optimization

A Step-by-Step Study Guide & Coding Workbook for Beginners

Welcome Student! This study workbook is designed to take you from a complete beginner in quantitative finance to someone who understands data collection, feature engineering, neural network training, and mathematical portfolio optimization. Follow this guide module-by-module to master the principles of building modern financial algorithms.

Module 1: Environment & Setup

To build financial models, we need a programming environment. We use **Python 3** and a list of open-source packages. Let's learn what these packages do:

- **pandas**: Used to load, clean, slice, and pivot tabular time-series datasets.
- **numpy**: Handles fast vector and matrix calculations (arrays).
- **yfinance**: Fetches historical stock data directly from Yahoo Finance API.
- **scikit-learn**: Handles data scaling, splitting, and PCA dimensionality reduction.
- **tensorflow**: The deep learning framework used to construct and train the LSTM model.
- **scipy**: Holds optimization solvers to find the optimal stock portfolio weights.

Try it Yourself (Exercise):

Install these dependencies using pip in your terminal:

```
pip install pandas numpy scikit-learn tensorflow scipy yfinance matplotlib notebook
```

Module 2: Downloading & Cleaning Data

A quantitative model is only as good as its data. We query historical daily prices for 61 stocks. Financial APIs can sometimes drop connections or block requests if called too fast.

What you will learn:

- **OHLCV**: Open, High, Low, Close, Volume. These are the 5 standard daily metrics for any asset.
- **Rate Limiting**: We add `time.sleep(1.0)` between requests so the Yahoo Finance server does not throttle or block our IP address.
- **Pivoting**: Extracting the `Close` prices for all stocks and storing them as a single clean grid indexed by `Date`.

Coding Challenge:

Write a Python function to read `stock\_new.csv` and extract only the Close prices of 'AAPL' and 'MSFT'. Save this pivoted dataframe as a new CSV file.

Module 3: Preprocessing & Technical Features

Raw stock prices cannot be fed directly into machine learning models because they are non-stationary. Instead, we calculate normalized changes (features).

Understanding the Mathematics:

- **Daily Returns:** $R_t = (C_t - C_{t-1}) / C_{t-1}$. Measures daily price movement.
- **20-Day Volatility:** Rolling standard deviation of daily returns over the last 20 days. Measures risk.
- **Z-Scores:** Formula: $Z_t = (value_t - mean_{\{rolling\}}) / std_{\{rolling\}}$. Shifts values to have a mean of 0 and standard deviation of 1. We shift the rolling mean and std by 1 day (`shift(1)`) to avoid data leakage.

Try it Yourself (Exercise):

Compute a 20-day simple moving average of Close prices in pandas. Hint: use `df['Close'].rolling(20).mean()`.

Module 4: Machine Learning & PCA-LSTM

LSTMs (Long Short-Term Memory) are deep learning networks capable of remembering long-term historical context for forecasting.

Key Machine Learning Concepts:

- **PCA (Principal Component Analysis):** Compresses the 671 features down to 61 components, removing multicollinearity (correlated inputs) and noise.
- **Sequence Generation:** We take a 252-day lookback window of features to predict a 22-day horizon of future prices.
- **LSTM Layers:** Two LSTM layers (800 and 600 units) extract sequential patterns, and Dense layers map them to the 22-day forecasted prices.

Coding Challenge:

Write a TensorFlow Sequential model with an Input layer of shape `(252, 61)` and a single LSTM layer of 128 units.

Module 5: Portfolio Optimization & Allocation

Once we have the predicted returns for the next month, we need to decide how much money to invest in each stock. We use mean-variance optimization (Markowitz Portfolio Theory).

Objective & Regularization:

We maximize portfolio return and minimize portfolio variance, subject to constraints. We also add an **L1-Regularization (Lasso)** penalty to the optimizer. The L1 penalty forces the weights of less optimal stocks to exactly zero. This creates a **sparse portfolio** consisting of only the top 5-10 best stocks, which is highly practical as it lowers transaction costs.

Constraints:

- **Long-only:** Weights must be between 0 and 1 (no short-selling).
- **Fully invested:** Weights must sum to 1 (all cash is allocated).

Try it Yourself (Exercise):

Run the SciPy minimize function on a basic quadratic function $f(x) = (x-3)^2$ to find its minimum. Verify that it converges to $x=3$.

Module 6: Backtesting & Performance Analysis

A backtest simulates how the model would have performed historically. We rebalance the portfolio monthly over a 5-year rolling window.

Analyzing the Output:

- **Compounded Return:** We start with \$100 and compound actual monthly returns using $\text{equity}_t = \text{equity}_{t-1} \times e^{r_t}$.
- **Sharpe Ratio:** Measures risk-adjusted return. Formula: $\text{Sharpe} = R_p / \sigma_p$ (return divided by standard deviation). A higher Sharpe ratio indicates better return per unit of risk.

Study Check (The Sharpe Bug):

Explain why dividing the return by the standard deviation of a scalar monthly variance (which is always 0) leads to an error. How did we resolve this in our codebase?